

Introduction & Project Objective

The purpose of this project is to build a **Sentimental Analysis program** that determines whether a user's input text expresses a **Positive**, **Negative**, or **Neutral** sentiment.

The program analyzes sentences entered by the user and calculates a polarity score based on emotional intensity, negation words, and sentiment-bearing keywords.

Problem it Solves

People often need quick ways to identify the emotional tone of statements, such as:

- Customer reviews
- User feedback
- Social media comments
- Surveys and forms

This program automates that process.

Intended Users

Anyone who needs simple sentiment classification, especially beginners learning Natural Language Processing (NLP).

Methodology / Code Explanation

1. Program Structure

The program consists of the following components:

- Importing Libraries: Uses nltk and regular expressions.
 - Sentiment Lists: Predefined lists of negation words and intensifiers to modify the score.
 - Main Function (analyze): Computes sentiment using both VADER and custom rules.
 - Interactive Loop: Allows users to repeatedly input text.
-

2. Key Functions

`analyze(txt)`

This is the core function of the program. It:

- Calculates VADER's built-in compound score
- Breaks the text into words
- Applies intensifiers (e.g., "extremely", "very")
- Detects negation words (e.g., "not", "never")
- Adjusts the sentiment score for each sentiment-bearing word
- Computes final polarity
- Returns:
 - Sentiment label (Positive/Negative/Neutral)
 - Polarity value
 - Keywords contributing to the sentiment

3. Libraries Used

Library	Usage
<code>nltk.sentiment.vader</code>	Provides the base sentiment analysis model (VADER).
<code>re</code>	Used to clean words by removing punctuation.
<code>nltk.download()</code>	Ensures the VADER lexicon is available.

These allow the program to detect sentiment patterns and process text efficiently.

4. Logic & Flow of the Program

1. User enters text
2. Program cleans & splits the text
3. VADER scores each word
4. Check for intensifiers within 2 previous words
5. Check for negation within 3 previous words
6. Modify the score accordingly
7. Average score + VADER compound score → final polarity
8. Classify sentiment:
 - Polarity > 0.05 → Positive
 - Polarity < -0.05 → Negative
 - Otherwise → Neutral
9. Display the results

This hybrid method combines traditional VADER sentiment analysis with custom rule-based logic to improve accuracy.

Results & Output

Below is an example of how the program runs in the terminal:

Example 1:

```
-> The Sun Revolve Around the Sun  
  
--- Result ---  
Sentiment: Neutral  
Polarity: 0.0  
-> █
```

Example 2:

```
-> The product is good but the delivery was terrible.  
  
--- Result ---  
Sentiment: Negative  
Polarity: -0.256025  
  
Key Words:  
• good(0.4404->0.4404)  
• terrible.(-0.4767->-0.4767)  
-> █
```

Example 3:

```
-> "The food was amazing, but the service was not great."  
  
--- Result ---  
Sentiment: Negative  
Polarity: -0.24285  
  
Key Words:  
• amazing,(0.5859->0.5859)  
• great.(0.6249->-0.6249)  
-> █
```

Challenges & Solutions:

Challenge	Solution
Understanding how VADER works internally	Researched documentation and experimented with sample text
Handling negations properly	Added custom list of negation words and reversed sentiment values when detected
Dealing with intensifying words like “very”, “extremely”	Created a dictionary of intensifiers with different multiplier strengths
Combining VADER score with custom logic	Computed weighted average of built-in and custom scores

Conclusion & Learning Outcomes

This project successfully implements a Sentimental Analysis tool using Python, VADER, and custom NLP rules.

It demonstrates how text preprocessing, negation handling, and intensity modifiers influence sentiment classification.

What I Learned

- How to use the NLTK VADER sentiment analyzer
- How to apply rule-based sentiment corrections
- How intensifiers and negations change emotional meaning
- How to interactively run Python programs in a loop
- How to combine multiple scoring methods to improve results